

Balancing Workload Performance and Slurm Stress: Four Nextflow Deployment Strategies

Nil Tianchen Mu*, William Dizon*, Glen Otero*, Torey Battelle*

*Research Computing, Arizona State University, Tempe, AZ, USA

Email: {nilmu, wdizon, gotero3, tbattell}@asu.edu

Abstract—Wide Nextflow fan-outs on shared Slurm clusters can submit tens of thousands of short tasks. Deployment settings route them through individual jobs, arrays, or nested schedulers inside enclosing allocations. They determine both workflow turnaround and RPC volume, a shared cost that can degrade scheduler responsiveness for every user. Existing comparisons evaluate whole workflow systems, while per-task queueing metrics cannot span architectures that dispatch inside an existing allocation.

This paper contributes a reproducible measurement protocol and bench harness that combine two elements. A clean-start clock begins before any backend service or allocation request, which places architecturally different backends on one comparable time axis. A disciplined use of per-user Slurm `sdiag` counters attributes request count as the primary *RPC demand* measure and controller processing time as sensitivity context, separate from the cluster-wide state that every co-tenant shares. The method is instantiated on four multi-node deployment strategies: Slurm native, Slurm job arrays, HyperQueue, and Flux. All four are measured on the shared ASU Phoenix production cluster; the single-user Dev campaign reports native, arrays, and Flux, with HyperQueue excluded. The fixed input contains 4,823 LASTZ tasks. The present preliminary campaign reports one clean-start trial per configuration. Its observations show a trade-off between walltime and RPC demand. On Phoenix, job arrays and HyperQueue reached the milestone in 0.32 h, while Flux required 0.65 h but reduced attributable RPC count to 1,396 per 1,000 terminal tasks; all three were non-dominated under the count metric. On Dev, arrays and Flux formed the observed frontier: arrays completed in 0.80 h, while Flux required 0.84 h and reduced attributable RPC count to 1,769 per 1,000 terminal tasks. The method lets an HPC site evaluate workflow deployment strategies with metrics that reflect both user experience and scheduler impact, and select the fastest strategy within its own limit on RPC demand.

Index Terms—Workflow management systems, Nextflow, HPC, Slurm, Fairshare, job scheduling, HyperQueue, Flux, meta-scheduler, benchmarking, reproducibility

I. INTRODUCTION

A standardized nf-core pipeline does not prescribe how it reaches Slurm. Nextflow can submit individual jobs, form arrays, or delegate dispatch to HyperQueue [5] or Flux [6] inside enclosing allocations. This small configuration choice controls both turnaround and RPC demand, yet is generally made by convention rather than measurement.

We measure workflow performance and Slurm control-plane demand for standardized Nextflow [1] and nf-core [2] pipelines on Slurm [3]. The target class has a dominant fan-out of short independent tasks; whole-genome alignment can contain about 40,000 such tasks [33]. Task core-hours and

CPU utilization provide context, while reserved idle capacity and cluster fragmentation are out of scope.

On a shared, Fairshare-scheduled cluster this fan-out has two coupled effects that the WMS literature has largely separated. Every native task adds a submission RPC and repeated status RPCs. Separately, the initial wave consumes enough TRES usage to drive the association’s Fairshare factor toward zero, after which later tasks progress mainly through back-fill [33], [34]. Section VI-A separates the two mechanisms. A deployment can therefore improve user time while worsening RPC demand, or relieve Slurm by paying for a larger enclosing allocation. Each choice also changes acquisition delay, multi-node reach, accounting visibility, policy enforcement, and failure isolation.

Which choice is right remains site- and workload-dependent; Section II-C positions this measurement question against prior workflow, pilot-job, and scheduler work.

This paper contributes the measurement method for that question, and characterizes the Nextflow-to-Slurm dispatch interface with it. Two obstacles make the question hard to measure. Per-task queueing metrics [27] cannot compare a Slurm task to a task dispatched inside an allocation that already exists, because the two wait in different places. A cluster-wide scheduler reading, in turn, cannot say which co-tenant caused it, so it cannot price one deployment’s demand. Each element of the method removes one obstacle:

- **C1: a clean-start protocol.** A common clock opens before backend initialization and before the first Slurm request, placing direct and allocation-backed strategies on one walltime axis.
- **C2: disciplined use of existing Slurm instrumentation.** Per-user `sdiag` counters, a separate observer identity, boundary snapshots, and an RPC-free settle tail attribute one deployment’s requests and observed controller processing time separately from cluster-wide Slurm state.
- **C3: an empirical application.** We apply the method to four deployment strategies on private and production Slurm clusters, exposing the observed trade-off between workflow performance and attributable RPC demand.

II. BACKGROUND AND RELATED WORK

A. Nextflow executors on Slurm

Nextflow hands each process invocation to an *executor* that places it on a backend. We select four strategies spanning per-

task dispatch, Slurm-native aggregation, and nested schedulers (Table I) [5]–[7].

Terminology. We call the long-running Nextflow driver the *main job*, and each Slurm job it submits per pipeline task a *per-task job*. We avoid “child job”: at the Slurm level these are independent submissions, not scheduler-tracked dependents. Every strategy submits one main job, so Table I lists it beside the further Slurm objects each strategy creates. For Flux, K denotes the fixed node count of its enclosing allocation; Table II gives K for each cluster.

B. Meta-schedulers on Slurm

HyperQueue obtains Slurm worker allocations and dispatches tasks through an internal server, whereas Flux starts a hierarchical broker inside an enclosing Slurm allocation [5], [6]. In both cases Slurm accounts for enclosing resources but cannot control internal tasks separately; aggregation changes accounting granularity rather than evading Fairshare.

C. Positioning against existing benchmarking infrastructure

WfCommons [19] supplies complementary synthetic-workflow infrastructure; we use a real standardized pipeline so task and request distributions are observed. ExaWorks [21] integrates large-scale workflow technologies, while prior bioinformatics comparisons [28], [29] compare whole WMS.

Dispatch inside a granted allocation is the pilot-job pattern [22]; Parsl [23] and Balsam [24] likewise aggregate task streams behind fewer batch jobs.

Slurm architecture and guidance describe aggregation benefits and message-, scheduling-, and lock-sensitive throughput limits [12], [18], [25]; real-machine simulator validation adds scheduler-level evidence [26]. They do not attribute controller demand to one workflow identity across dispatch strategies under production co-tenancy. We measure that narrower trade-off.

III. MEASUREMENT METHOD AND STUDY HARNESS

The harness makes the experimental clock and attribution rules reproducible in three parts.

Clean-start backend adapters. Each configuration has a lifecycle adapter that starts without retained allocations or services, initializes the strategy, waits for the milestone, validates correctness, and releases every trial-owned Slurm object. Only adapters are backend-specific.

Metric collectors. Monitoring runs under the separate Slurm-root identity on the scheduler host, outside the measured allocation. Boundary `sdiag` snapshots supply the bench user’s cumulative RPC count and processing time, whose boundary differences produce the two attributable RPC measures; a 300 s series records their evolution and cluster context (Figure 1). Observer RPCs are attributed to Slurm root. Because `sdiag` itself issues an RPC [13], the collector limits bench-user monitoring traffic to one `squeue` before the pre-boundary, outside the measured window; low-rate `sdiag` sampling and drain checks run as the separate observer. Task timing and identifiers come from the Nextflow trace [9];

Slurm-root markers record the allocation-inclusive start t^0 and release boundary. Raw trace, telemetry, provenance, and correctness artifacts are retained.

Reporting. Offline scripts produce milestone walltimes, per-user RPC deltas, cluster-wide state, and the trade-off of Section V.

IV. EXPERIMENTAL DESIGN

A. Backends under test

The four backends of Table I run identical pipeline code under the settings of Table II. Site-specific operating choices are frozen separately for each treatment before its first accepted run and draw on prior scheduler-friendly practice [32]; this paper does not estimate an optimum for those choices. Because the backends differ in resource ownership and dispatch locus, the treatments are *deployment strategies* rather than executors differing in a single implementation detail.

Matching rule. Within a cluster, runs use the same pipeline and input, bench user, account, QOS, clean-start boundary, and completion gate. Placement, per-task limits, and allocated concurrency are frozen per treatment rather than matched: they are part of the deployment strategy. On Phoenix, native, job-array, and HQ dispatch can reach the 300-node partition capacity, with HQ requesting one-node workers on demand up to 300. Flux instead requests 32 fixed nodes at once; a 300-node preallocation would be unrealistic. The comparison therefore estimates deployable strategies, not backend effects at equal concurrency. Each treatment uses Section III’s clean-start lifecycle.

Local execution is supported by the harness but excluded because one enclosing node cannot represent scalable execution across an HPC cluster.

The Flux backend carries one documented fidelity caveat. The Nextflow Flux executor does not support the per-process memory directive [8], so its runs cannot reproduce the memory requests that the other backends honor. Slurm instead reserves all memory on the K complete nodes for Flux, while Flux does not use the common per-task memory request for internal admission. The caveat is reported with the results rather than excluded, because a backend’s constraints are part of what an operator is choosing between.

B. Reference workload and its descriptors

The protocol is not tied to the reference pipeline. Applying it to another stage-limited Nextflow pipeline requires no process-name allowlist: analysis infers the terminal process from the latest successful trace completion. We report the driving workload through descriptors another site can use for comparison rather than by name alone. The class is *wide fan-out*: a small serial preparation stage followed by one dominant parallel stage of many short, independent tasks. Four descriptors characterize it. Endpoint-stage cardinality places the example within the approximate scale range reported in Table II; every run contains 4,823 distinct successful LASTZ logical tasks, followed by a 91-task CAT_PSL terminal stage, and the structural DAG depth is 4. Observed Phoenix task

TABLE I: Four multi-node deployment strategies. Their Slurm objects define the dispatch, control, visibility, and TRES-accounting units.

Backend	Executor	Slurm objects created	Dispatched by	Slurm-level control
Slurm native	slurm	main job + one job per task	Slurm	per task
Slurm job array	slurm+array	main job + one array job per batch	Slurm	per element
HyperQueue	hq	main job + HQ-requested worker allocations	HQ server	outer only
Flux	flux	main job is the K -node allocation, plus one srun step	Flux brokers	outer only

TABLE II: Experimental settings. Values are frozen per cluster and treatment.

Factor	Dev	Phoenix
Deployment backend	four levels (Table I)	
Software	Nextflow 26.04.0; HyperQueue 0.26.2; Flux 0.88.0; Apptainer 1.4.5-3.el8; container nilablueshirt/make_lastz_chains:latest-amd64	
Workload and request	frozen pipeline, small input producing 4,823 LASTZ logical tasks; production inputs span approximately 5,000–200,000; 1 CPU and 4 GB per task; 15 min per-task base time limit	
Slurm executor settings	pollInterval=10s; queueStatInterval=1min; submitRateLimit=100/1min; exitReadTimeout=5min; killBatchSize=10000	
Slurm native/array Allocation backends	queueSize=600; arrays of 100 HQ: partial-node driver allocation and 4-worker ceiling (at most 3 exclusive workers concurrent while the driver is active); Flux: $K = 4$ fixed nodes (512 CPU slots)	queueSize=10000; arrays of 500 HQ: demand-driven, at most 300 one-node workers; Flux: $K = 32$ fixed nodes (896 CPU slots)
Platform	public; Slurm 26.05.1; 4 unified nodes; 128 CPUs and 512 GB RAM per node	public; Slurm 25.11.3; 300 unified nodes; 28 CPUs and 250 GB RAM per node
Slurm controller limits Scheduler policy/parameters	MaxArraySize=200000; MaxJobCount=400000 preempt_strict_order, nohold_on_prolog_fail, bf_window=40000, bf_resolution=300, bf_max_time=60, bf_max_job_user=1000, bf_max_job_test=100000, bf_continue, bf_interval=60, max_rpc_cnt=1000	MaxArraySize=300000; MaxJobCount=550000 preempt_strict_order, nohold_on_prolog_fail, bf_window=43200, bf_resolution=300, bf_max_time=120, bf_max_job_assoc=2600, bf_max_job_start=1300, bf_max_job_test=200000, bf_continue, bf_interval=30, max_rpc_cnt=450, enable_user_top
Controlled identity/state	bench user, account and QOS fixed per site; placement fixed per backend; RawUsage reset and verified zero	
Co-tenant background	none	production co-tenancy via cluster-wide sdiag

concurrency peaked at 446 for native, 4,362 for arrays, 4,777 for HQ, and 841 for Flux. Across those four traces, successful LASTZ tasks had a pooled median *realtime* of 255.7 s (IQR 146.5–342.5 s); every LASTZ task requests 1 CPU and 4 GB. Dispatch aggregation is set by the backend (Table I). These descriptors define the region of workload space represented here; additional pipelines are needed to determine how well they predict backend behavior elsewhere.

The instance is `make_lastz_chains` [30], a whole-genome pairwise alignment pipeline built on LASTZ [31] and the UCSC Kent toolkit, in the DSL2 form we refactored according to *nf-core* conventions in prior work [32], [33]. We use it as the reference workload because its Fairshare and backfill behavior on a shared cluster is already characterized, allowing the present study to focus on the dispatch interface. That prior work reported the native–array walltime split; new here are the reusable clean-start protocol and bench harness, the four-strategy comparison, and exact per-user boundary attribution. The tabulated milestone isolates the wide fan-out whose dispatch behavior the study measures, and truncating there keeps the four-backend campaign on two clusters within a feasible time and resource budget. The stage limit makes LASTZ behave as the natural end of a single-stage pipeline: every strategy completes the same scientific work and then performs its normal shutdown. The trace parser then identifies the corresponding successful task record, so the milestone is a completed unit of scientific work rather than a log line.

C. Clusters and partitions

We run the campaign on two Slurm clusters with different tenancy models. Their releases, selected partition geometry, and scheduler policies are reported in Table II. Both releases expose the `sdiag` per-user RPC table, allowing the collector to extract the dedicated bench user on each cluster.

Dev is a private, four-node, single-user cluster and provides the low-background reference. Daemon activity can still appear in its global counters; only the bench user’s per-user delta is directly attributable. *Dev*’s nondefault backfill settings permit deep queue examination and continued scheduling across lock yields while trading scheduling frequency and precision for lower overhead [17].

Phoenix is the ASU production cluster used in this study, a high-throughput machine tuned for workloads of many small tasks, with Slurm, Fairshare [4], and the backfill scheduler. Its scheduler is tuned to admit high per-association backfill volume and elevated RPC concurrency, which is what lets it absorb WMS fan-out; the relevant `SchedulerParameters` and the rationale are documented in [32], [34]. Both clusters are operated by ASU Research Computing, which also operates the separately documented Sol supercomputer [35].

We preserve rather than synchronize these site policies: treatment comparisons are controlled within each cluster, while *Dev* and *Phoenix* are contextual applications rather than a paired estimate of cluster effects.

Exact trial intervals are retained in each run’s immutable

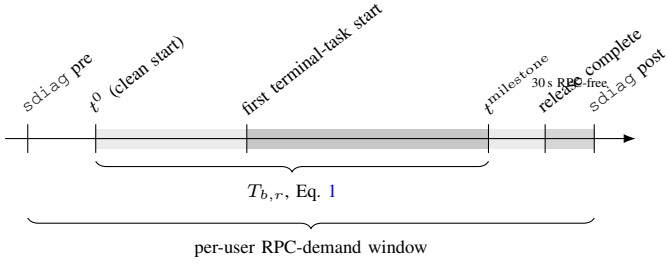


Fig. 1: Measurement boundaries for one trial. The clock opens at t^0 , before any backend service or allocation request, so backend startup and allocation acquisition occupy the first light band and count against every strategy. The dark band is task dispatch and execution. The closing light band is the correctness gate, main-job exit, and release; the final shaded band is the fixed 30-second RPC-free settle tail. The per-user RPC window brackets the whole lifecycle: activity after $t^{\text{milestone}}$ is excluded from walltime but included in RPC demand.

run.json, trial.env, and append-only events.tsv. These artifacts also record the command-line backend order, bench user, sampling cadence, reset requirement, and workload/resource hashes.

D. Metrics

Each run reports walltime, attributable per-user RPC demand, cluster-wide Slurm state, time to the first terminal-process task start, task core-hours, CPU utilization, and correctness. Slurm task control is not measured per run: it follows from the dispatch unit (Table I). Task core-hours sum requested CPUs times `realtime` over trace rows, and CPU utilization is the `realtime`-weighted percentage of requested CPUs. The page-limited paper uses two explicit equations. Our walltime boundary starts at the user action before backend startup or the first allocation request:

$$T_{b,r} = t_{b,r}^{\text{milestone}} - t_{b,r}^0. \quad (1)$$

For backend b and replicate r , t^0 precedes the first backend action and $t^{\text{milestone}}$ is completion of the final successful task at the common endpoint. The semantic validator runs after $t^{\text{milestone}}$ and does not enter $T_{b,r}$; a run enters the comparison only after that gate passes. The primary RPC-demand metric is normalized request count:

$$C_{b,r} = 1000 \frac{\Delta \text{RPCcount}_{u,b,r}}{L_{b,r}^{\text{endpoint}}}, \quad (2)$$

where `sdiag`'s per-user table supplies the cumulative count for bench user u [13], and $L_{b,r}^{\text{endpoint}}$ is the number of distinct successful logical tasks in the inferred endpoint process. The sensitivity metric $P_{b,r}$ replaces $\Delta \text{RPCcount}$ in Eq. 2 with cumulative RPC processing time converted from microseconds to seconds. Figure 1 places both boundaries. The pre-boundary is taken immediately before t^0 , and the post-boundary after a fixed 30-second release-settle tail in which the observer issues

no RPC. Allocation acquisition, dispatch, and the workflow-release tail are therefore included without repeated bench-user status polling. For the present fixed input, $L_{b,r}^{\text{endpoint}} = 91$ CAT_PSL tasks in every run, so normalization is a constant rescaling that does not change within-cluster ordering; it also provides a rate for future comparisons across input scales. The terminal process and task count are derived from each trace rather than configured as process-name keywords.

RPC count is structural demand attributed to the bench user. Processing time is controller execution under the conditions present during the run; contention, queues, and lock availability make it a sensitivity metric rather than an intrinsic backend property. Table III reports both, while successive 300 s samples provide bench and non-bench RPCs/min with the observer excluded.

The two primary axes intentionally use different endpoints. Walltime ends at the scientific milestone, whereas RPC demand continues through correctness checking, backend tear-down, and return to a clean state, capturing the deployment's complete Slurm-facing lifecycle through the fixed 30-second settle tail.

Cluster-wide `sdiag` readings are secondary context. Cycle-count differences give main and backfill cycles/min; sampled last-cycle durations, server threads, and agent queues describe non-attributable system conditions.

E. Sequential execution and cross-run comparability

The operator supplies a randomized backend order, which the harness executes sequentially and records. Each backend run is the experimental unit; samples within its `sdiag` series are repeated telemetry, not independent replicates.

The automated protocol accepts any positive number of independent repetitions; this paper uses one run ($N=1$) per configuration due to time constraints. Although the evaluated pipeline is deterministic in its tasks and outputs, system-level measurements may vary across runs, so the reported results should be considered preliminary. Results accordingly show absolute observations first; any native-normalized ratio is descriptive and does not control for temporal variation. Repetitions are independent. The two cluster campaigns may overlap in time, since the clusters are physically separate; each preserves its rule of one treatment at a time.

Before every run, the bench user has no pending or running jobs. An administrator resets that user's `RawUsage` in the tested account before t^0 , verifies the association at zero with `sshare`, and records the resulting `Fairshare` factor. Resource use by the initial wave and its effect on later tasks are therefore part of the treatment. Sites with contributing parent associations must instead isolate or match the effective starting `Fairshare` [14], [15].

Runs are scheduled not to cross `sdiag`'s default midnight-UTC statistics reset [13]. Any administrative reset or Slurm restart inside a run invalidates its RPC delta. The scientific task graph is deterministic, and clean-start execution leaves no service, worker, allocation, or workflow state for the next run. A run invalidated by infrastructure or execution

failure is archived and rerun independently from a fresh state. The campaign reports only runs that successfully complete Nextflow and provide a readable trace and report.

V. RESULTS: WALLTIME AGAINST RPC DEMAND

Every reported strategy must pass the same completion gate (successful Nextflow completion plus readable trace and report); a strategy that does less work is not faster. Table III gives the observation for each completed cell. Figure 3 plots all completed treatments on each cluster. The two displayed objectives remain separate because a site’s acceptable RPC demand is a policy choice.

A. Walltime and RPC demand

Walltime reports the quantity the researcher experiences. On Phoenix, job arrays and HQ completed in 0.322 and 0.321 h, respectively, 68.8% sooner than native’s 1.030 h. Flux completed in 0.647 h: slower than arrays and HQ, but 37.2% sooner than native. The first-terminal-task column separates time to the first terminal-process start from the remaining interval without pretending that an allocation-backed task and a Slurm-native task have the same queue. Its Phoenix values—0.308 h for arrays, 0.319 h for HQ, 0.637 h for Flux, and 1.016 h for native—show that the interval before the terminal process accounted for most of each observed walltime. That interval includes allocation acquisition, queueing, and earlier pipeline work.

On Dev, arrays completed in 0.800 h and Flux in 0.843 h, 20.5% and 16.2% sooner than native’s 1.006 h. Their first-endpoint-task delays were 0.786, 0.833, and 0.992 h, respectively. The Dev HQ run had not completed after 2.87 h—already $2.9\times$ native’s completed walltime—and was operator-stopped as operationally unreasonable for this four-node configuration; it is therefore excluded rather than treated as a censored completion.

The measured per-user deltas show that aggregation changed RPC demand in this campaign. Native issued 5,126 attributable RPCs, or 56,330 per 1,000 terminal tasks. Arrays issued 387 (4,253 per 1,000), HQ 415 (4,560), and Flux 127 (1,396), reductions of $13.2\times$, $12.4\times$, and $40.4\times$ relative to native. Processing-time sensitivity followed the broad reduction but not the exact count ordering: arrays, HQ, and Flux used 503.5, 448.2, and 23.4 s per 1,000 tasks, respectively, versus 8,008.5 for native. On Dev, native issued 5,120 attributable RPCs (56,264 per 1,000 terminal tasks), arrays 353 (3,879), and Flux 161 (1,769). Arrays and Flux therefore reduced count by $14.5\times$ and $31.8\times$ relative to native. Their processing-time sensitivities were 11.3 and 0.2 s per 1,000 tasks, respectively, versus 166.7 for native. Task core-hours and CPU utilization give active-work context; the fixed allocation geometries in Table II separately expose reserved capacity. Phoenix task execution consumed 305.0–358.8 core-hours across the four runs. Runtime-weighted CPU use was 96.2% for native, 99.0% for HQ, and 96.2% for Flux; the array estimate was 89.1% over the 92.3% of task core-hours with a reported CPU metric. On Dev, native, arrays, and Flux consumed 288.4, 331.2, and

290.3 task core-hours, with runtime-weighted CPU use of 95.4%, 84.3%, and 94.9%, respectively; all three had 100% metric coverage.

B. Phoenix cluster-wide Slurm state

Figure 2 shows sustained co-tenant activity: the run-median non-bench rate remained approximately 670–750 RPCs/min across all four windows and exceeded the bench-user rate in every run. The global scheduler context was not matched across treatments. Main-scheduler frequency was approximately 110–135 cycles/min during native and array runs, versus about 11 for HQ and 3 for Flux; backfill frequency stayed near 1.2–1.4 cycles/min. The temporal p95 agent queue remained near zero, while server-thread p95 ranged from about 2 to 15. The tested backend can itself affect global scheduling metrics, so overlap is context, not proof of identical background.

C. The non-dominated operating points

Figure 3 visualizes all completed treatments on each cluster. Strategy *A* dominates *B* only if it is no slower and issues no more attributable RPCs per 1,000 tasks, with at least one strict improvement [20]; we apply the test to the observed per-backend points. On that basis we identify the observed Pareto frontier: the set of strategies for which no other evaluated strategy is both faster and lower in attributable RPC count. On Phoenix it contains job arrays, HQ, and Flux. Arrays and HQ differ by only 3.3 s in walltime; arrays have the lower count, while HQ is slightly faster, so neither dominates the other. Flux is slower but has the lowest count. Native is dominated by all three. Relative to native, the observed (walltime, count) ratios are (0.312, 0.075) for arrays, (0.311, 0.081) for HQ, and (0.628, 0.025) for Flux. On Dev, arrays and Flux form the observed frontier, with respective (walltime, count) ratios to native of (0.795, 0.069) and (0.838, 0.031); native is dominated by both. Points that trade one objective against the other are alternatives for different site policies, not statistical ties or failures to identify a winner. Under the matching rule of Section IV-A, allocated concurrency is an explicit component of each operating point. It would confound a claim about backend implementation at equal capacity, but that is not the estimand here: the frontier compares deployable strategies, including the allocation acquisition time caused by their realistic resource geometries.

The accumulation panels resolve when the primary endpoint accrued, separating a low sustained request rate from a front-loaded fan-out. On Phoenix, native’s count rose in successive steps throughout its 1.03 h window. Arrays and HQ reached their final plateaus by approximately 0.34 h, whereas Flux accumulated smaller steps through approximately 0.66 h. The separate processing-time diagnostic shows the same temporal structure while preserving its different final ordering. Because the curves are per-user `sdiag` deltas rather than trace-derived submissions, they also cover the two backends that expose no per-task Slurm record. As a sensitivity test, we recompute

TABLE III: Primary results by cluster, $N=1$. Cells give the single observation per configuration. Walltime includes backend startup and allocation queueing. RPC-count cells give the total followed by the per-1,000-task value in parentheses; RPC seconds are normalized per 1,000 endpoint-stage tasks. Task core-hours and runtime-weighted CPU utilization come from the machine-readable Nextflow trace. CPU utilization is weighted over attempts with a reported CPU metric; metric coverage is reported with any incomplete cell. The dagger marks 92.3% CPU-metric coverage for the Phoenix job-array run; all other Phoenix and reported Dev runs have 100% coverage. These quantities describe task execution, not idle cores reserved in enclosing allocations. Dev HyperQueue was excluded after its noncompleted run was stopped at 2.87 h as operationally unreasonable for this configuration.

Backend	Clean-start h	First endpoint h	RPC count (/1k)	RPC s/1k	Task core-h	CPU use (%)
<i>Dev (single user)</i>						
Slurm native	1.006	0.992	5,120 (56,264)	166.7	288.4	95.4
Slurm job array	0.800	0.786	353 (3,879)	11.3	331.2	84.3
HyperQueue		N/A (stopped at 2.87 h; no completed run)				
Flux	0.843	0.833	161 (1,769)	0.2	290.3	94.9
<i>Phoenix (shared)</i>						
Slurm native	1.030	1.016	5,126 (56,330)	8,008.5	305.0	96.2
Slurm job array	0.322	0.308	387 (4,253)	503.5	358.8	89.1 [†]
HyperQueue	0.321	0.319	415 (4,560)	448.2	329.3	99.0
Flux	0.647	0.637	127 (1,396)	23.4	305.9	96.2

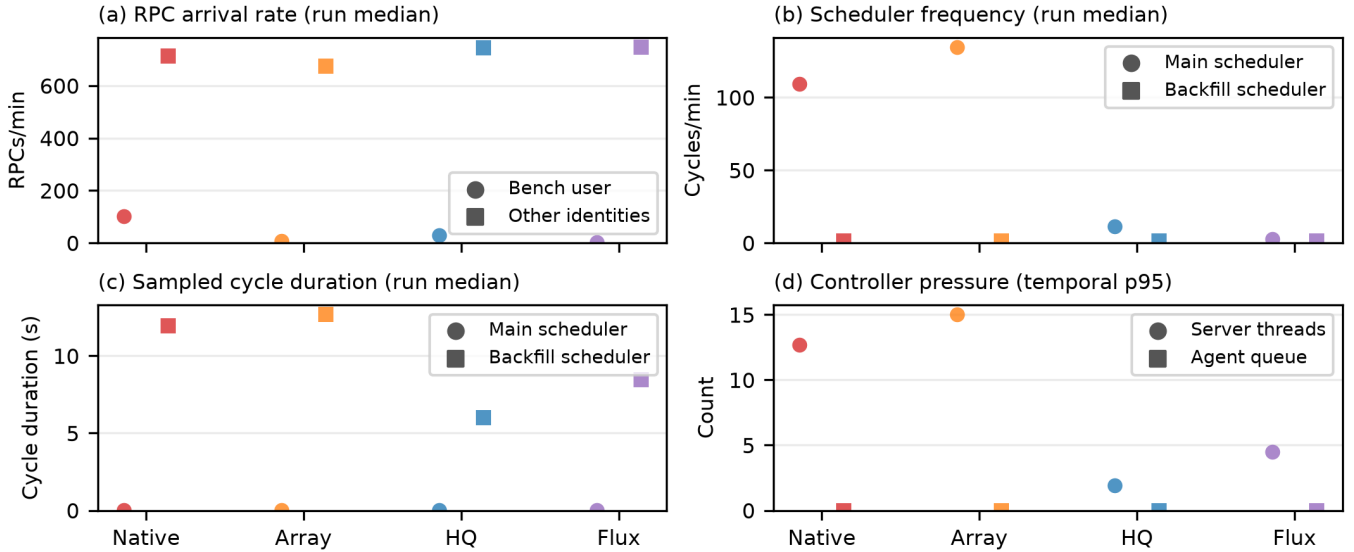


Fig. 2: Secondary Phoenix system context from 300 s samples ($N=1$). Panel (a) separates RPCs attributed to the bench user from other identities, excluding the observer. Panels (b)–(d) include co-tenant and Slurm activity and are not attributed solely to a backend.

dominance using processing time and find a narrow divergence: HQ is both slightly faster and lower in processing time than arrays, so the processing-time frontier contains HQ and Flux, while the primary count frontier also contains arrays.

VI. OPERATIONAL INTERPRETATION

A. Fairshare, backfill, and task geometry

Fairshare and RPC aggregation are distinct mechanisms. Fairshare reflects accumulated TRES usage rather than submission count [14], [16]. In prior Phoenix measurements, the initial LASTZ wave rapidly reduced the bench user’s association Fairshare, making later progress increasingly dependent

on backfill [33], [34]. Short, accurate resource requests therefore remain important regardless of dispatch backend [17].

Arrays preserve independently schedulable elements while amortizing submission and polling [7], [12]. Task geometry therefore controls TRES consumption and backfill fit, while array grouping primarily controls RPC traffic. That separation predicts a cluster-dependent effect, which is what earlier measurements show: arrays improved walltime on the shared cluster and not on the single-user one [32].

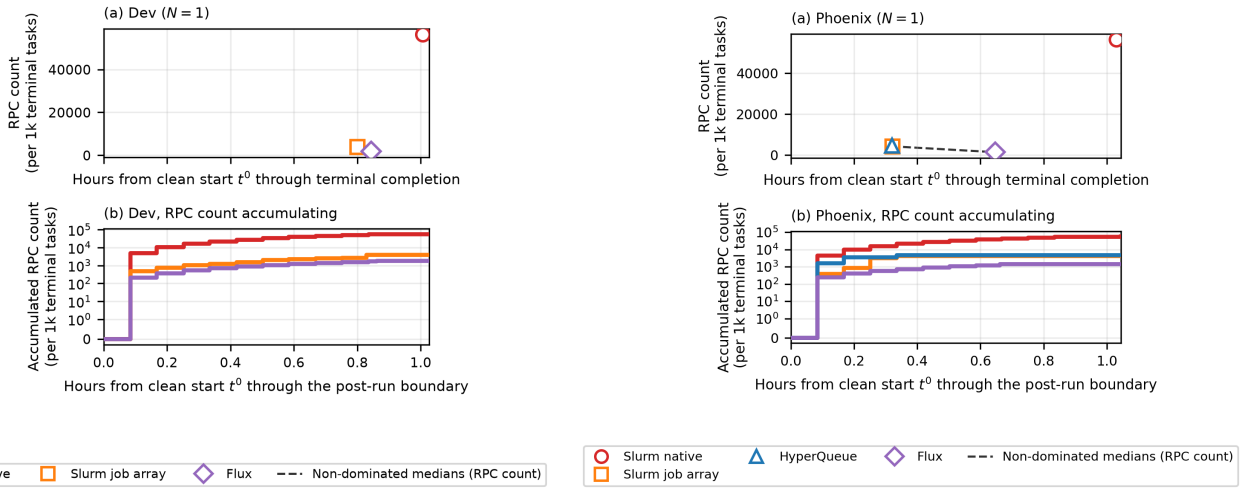


Fig. 3: Observed Dev (left) and Phoenix (right) performance–demand trade-offs ($N=1$). The Dev plot contains native, arrays, and Flux; the Phoenix plot contains all four completed treatments, including HQ. In each plot, panel (a) places walltime against attributable RPC count and panel (b) shows when requests accrued through the post-tail boundary. Lower-left is better. Each curve ends on the normalized count in Table III, which separately reports processing time as sensitivity context. This is one pipeline–input operating point, not a universal saturation curve.

B. Acquisition cost versus dispatch relief

All trials begin from a clean start, so the comparison is between patterns of resource acquisition rather than between reused and freshly acquired state. The dispatch units in Table I imply different acquisition costs. Table III’s first-terminal-task interval quantifies the up-front price, while total walltime shows whether faster in-allocation dispatch repays it by the milestone. This decomposition matters particularly on a shared partition: a long K -node request may wait longer than many small backfill-friendly requests even when its measured attributable RPC demand is lower.

VII. DISCUSSION

Applying the method. A site selects a representative input and correctness-checked milestone, records Section IV-B’s descriptors, and repeats Eqs. 1–2 under fixed policy with a separate observer. Sites unable to reset an isolated association should block or match starting Fairshare. They reject strategies violating demand, capacity, efficiency, or control constraints, then select the fastest remaining option. Aggregation hides per-task records, limits, cancellation, and preemption from Slurm, but the enclosing allocation still accrues aggregate TRES usage (Table I).

Generalization and threats to validity. The method targets standardized Nextflow execution, including nf-core pipelines, on university Fairshare-based Slurm. The campaign covers one region of the descriptor space, so it reports the backend ordering observed for this wide fan-out workload, cardinality, and task geometry rather than every DAG shape. The protocol applies where Slurm exposes per-user `sdiag` statistics and permits a separate observer. Rankings remain conditional on workload, hardware, policy, tenancy, and background. Phoenix background varies and $N=1$ cannot average it out; attribution

remains per-user, but global response includes co-tenants. The physically separate clusters are contextual applications, not a paired estimate of cluster effects. Equal starting Fairshare does not imply equal priority or ordinary user histories. The 4,823-task input is also substantially smaller than the roughly 40,000-task motivating case; total RPC demand and backend ordering must not be extrapolated to larger cardinalities from this campaign. Finally, this study evaluates freshly started services only; persistent HQ or Flux deployments may occupy different points.

VIII. CONCLUSION AND FUTURE WORK

We compared Nextflow deployment strategies using clean-start walltime and per-user Slurm RPC attribution on private and production clusters: three completed treatments on Dev and four on Phoenix. In the Phoenix observation, arrays and HQ reduced walltime by about 69% relative to native while reducing attributable RPC count by 92%; Flux reduced count by 98% but took about twice as long as arrays and HQ. On Dev, arrays and Flux reduced walltime by 20.5% and 16.2%, respectively, while reducing count by 93.1% and 96.9% relative to native. The results show that deployment choice trades allocation acquisition and task-level visibility against RPC aggregation and workflow turnaround; no strategy is optimal under every site constraint. The primary contribution is the reusable methodology that produced the observations rather than a universal ranking. A deployment choice inside a workflow manager is therefore also a scheduling decision for the site that hosts it.

Future work will repeat the campaign at $N > 1$ and several cardinalities, then extend it to other pipelines and workflow systems, beginning with Snakemake [10] and Pegasus [11]. It will sweep endpoint cardinality and DAG shape to test

the hypothesis that total RPC demand grows with task count and that dispatch aggregation becomes increasingly valuable, reporting both total and per-1,000-task demand. WfBench within WfCommons [19] will provide portable synthetic cases.

ARTIFACT AVAILABILITY

The released artifact contains the backend configurations, measurement harness, raw per-run records, and analysis scripts required to reproduce Table III and Figures 2 and 3, and supports repetition at arbitrary N without modification. Source and artifacts are at <https://github.com/NilaBlueshirt/Slurm-Stress-WORKS26-SC26>. The same repository carries the accepted author version of [32], whose publication of record is forthcoming.

ACKNOWLEDGMENTS

We thank the ASU Research Computing team for cluster access and operational support, the Earth BioGenome Project for the collaboration that motivated the reference workload, and the upstream `make_lastz_chains` authors at the Hiller Lab. We thank SchedMD, the HyperQueue and Flux teams for their tools.

REFERENCES

- [1] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, “Nextflow enables reproducible computational workflows,” *Nature Biotechnology*, vol. 35, pp. 316–319, 2017. doi:10.1038/nbt.3820.
- [2] P. A. Ewels *et al.*, “The nf-core framework for community-curated bioinformatics pipelines,” *Nature Biotechnology*, vol. 38, no. 3, pp. 276–278, 2020.
- [3] A. B. Yoo, M. A. Jette, and M. Grondona, “SLURM: Simple Linux Utility for Resource Management,” in *Job Scheduling Strategies for Parallel Processing (JSSPP 2003)*, LNCS 2862, Springer, 2003, pp. 44–60.
- [4] J. Yalim, “Toward dynamically controlling Slurm’s classic Fairshare algorithm,” in *Practice and Experience in Advanced Research Computing 2020 (PEARC ’20)*, ACM, 2020, pp. 538–542. doi:10.1145/3311790.3399628.
- [5] J. Beránek, A. Böhm, G. Palermo, J. Martinovič, and B. Janský, “HyperQueue: Efficient and ergonomic task graphs on HPC clusters,” *SoftwareX*, vol. 27, 101814, 2024. doi:10.1016/j.softx.2024.101814.
- [6] D. H. Ahn, J. Garlick, M. Grondona, D. Lipari, B. Springmeyer, and M. Schulz, “Flux: A next-generation resource management framework for large HPC centers,” in *43rd Int. Conf. on Parallel Processing Workshops (ICPPW)*, IEEE, 2014, pp. 9–17. doi:10.1109/ICPPW.2014.15.
- [7] Nextflow Project, “array directive — Nextflow process reference,” accessed 2026. <https://docs.nextflow.io/latest/reference/process>.
- [8] Nextflow Project, “Executors: Flux executor,” accessed Aug. 2026. <https://www.nextflow.io/docs/latest/executor.html#flux>.
- [9] Nextflow Project, “Reports: Trace file,” accessed Aug. 2026. <https://docs.nextflow.io/latest/reports>.
- [10] J. Köster and S. Rahmann, “Snakemake—a scalable bioinformatics workflow engine,” *Bioinformatics*, vol. 28, no. 19, pp. 2520–2522, 2012. doi:10.1093/bioinformatics/bts480.
- [11] E. Deelman *et al.*, “Pegasus, a workflow management system for science automation,” *Future Generation Computer Systems*, vol. 46, pp. 17–35, 2015. doi:10.1016/j.future.2014.10.008.
- [12] SchedMD, “Slurm job array support,” accessed Aug. 2026. https://slurm.schedmd.com/job_array.html.
- [13] SchedMD, “sdiag: Slurm scheduling diagnostic tool,” accessed Aug. 2026. <https://slurm.schedmd.com/sdiag.html>.
- [14] SchedMD, “sshare: Slurm association shares and usage,” accessed Aug. 2026. <https://slurm.schedmd.com/sshare.html>.
- [15] SchedMD, “sacctmgr: Slurm accounting management,” accessed Aug. 2026. <https://slurm.schedmd.com/sacctmgr.html>.
- [16] SchedMD, “Multifactor priority plugin,” accessed Aug. 2026. https://slurm.schedmd.com/priority_multifactor.html.
- [17] SchedMD, “Slurm scheduling configuration guide,” accessed Aug. 2026. https://slurm.schedmd.com/sched_config.html.
- [18] SchedMD, “High throughput computing administration guide,” accessed Aug. 2026. https://slurm.schedmd.com/high_throughput.html.
- [19] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, “WfCommons: A framework for enabling scientific workflow research and development,” *Future Generation Computer Systems*, vol. 128, pp. 16–27, 2022. doi:10.1016/j.future.2021.09.043.
- [20] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002. doi:10.1109/4235.996017.
- [21] A. Al-Saadi *et al.*, “ExaWorks: Workflows for exascale,” in *IEEE Workshop on Workflows in Support of Large-Scale Science (WORKS)*, 2021, pp. 50–57. doi:10.1109/WORKS54523.2021.00012.
- [22] M. Turilli, M. Santocroci, and S. Jha, “A comprehensive perspective on pilot-job systems,” *ACM Computing Surveys*, vol. 51, no. 2, article 43, 2018. doi:10.1145/3177851.
- [23] Y. Babuji *et al.*, “Parsl: Pervasive parallel programming in Python,” in *28th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC)*, ACM, 2019, pp. 25–36. doi:10.1145/3307681.3325400.
- [24] M. Salim, T. Uram, J. T. Childers, P. Balaprakash, V. Vishwanath, and M. Papka, “Balsam: Automated scheduling and execution of dynamic, data-intensive HPC workflows,” arXiv:1909.08704, 2019.
- [25] M. A. Jette and T. Wickberg, “Architecture of the Slurm workload manager,” in *Job Scheduling Strategies for Parallel Processing (JSSPP 2023)*, LNCS 14283, Springer, 2023, pp. 3–23. doi:10.1007/978-3-031-43943-8_1.
- [26] A. Jokanovic, M. D’Amico, and J. Corbalan, “Evaluating SLURM simulator with real-machine SLURM and vice versa,” in *IEEE/ACM PMBS*, 2018, pp. 72–82. doi:10.1109/PMBS.2018.8641556.
- [27] D. G. Feitelson and L. Rudolph, “Metrics and benchmarking for parallel job scheduling,” in *Job Scheduling Strategies for Parallel Processing (JSSPP)*, LNCS 1459, 1998, pp. 1–24. doi:10.1007/BFb0053978.
- [28] E. Larsson, J. Mercier, N. Wiart, E. L. Floch, O. Delhomme, and V. Meyer, “Evaluating workflow management systems: A bioinformatics use case,” in *IEEE Int. Conf. on Bioinformatics and Biomedicine (BIBM)*, 2018, pp. 2773–2775. doi:10.1109/BIBM.2018.8621141.
- [29] L. Wratten, A. Wilm, and J. Göke, “Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers,” *Nature Methods*, vol. 18, pp. 1161–1168, 2021. doi:10.1038/s41592-021-01254-9.
- [30] B. M. Kirilenko *et al.*, “Integrating gene annotation with orthology inference at scale,” *Science*, vol. 380, no. 6643, article eabn3107, 2023. doi:10.1126/science.abn3107.
- [31] R. S. Harris, *Improved Pairwise Alignment of Genomic DNA*, Ph.D. thesis, The Pennsylvania State University, 2007.
- [32] N. T. Mu, W. Dizon, G. Otero, and T. Battelle, “Ten practices for refactoring scientific pipelines to nf-core DSL2 on shared Slurm HPC,” in *US Research Software Engineering Conference 2026 (USRSE’26)*, 2026 (to appear). doi:10.5281/zenodo.21831314 (registered on publication).
- [33] N. T. Mu, W. Dizon, G. Otero, and T. Battelle, “Optimizing Nextflow-based software on shared HPC resources: A case study with `make_lastz_chains`,” in *US Research Software Engineering Conference 2025 (USRSE’25)*, Philadelphia, PA, USA, Sep. 2025. doi:10.5281/zenodo.17118383.
- [34] N. T. Mu, W. Dizon, G. Otero, and T. Battelle, “Tuning workflow management systems for shared HPC resources,” presented at the *Rocky Mountain Advanced Computing Consortium HPC Symposium (RMACC)*, Boise, ID, USA, May 2026. doi:10.5281/zenodo.20192500.
- [35] D. M. Jennewein *et al.*, “The Sol Supercomputer at Arizona State University,” in *PEARC ’23*, ACM, 2023, pp. 296–301. doi:10.1145/3569951.3597573.